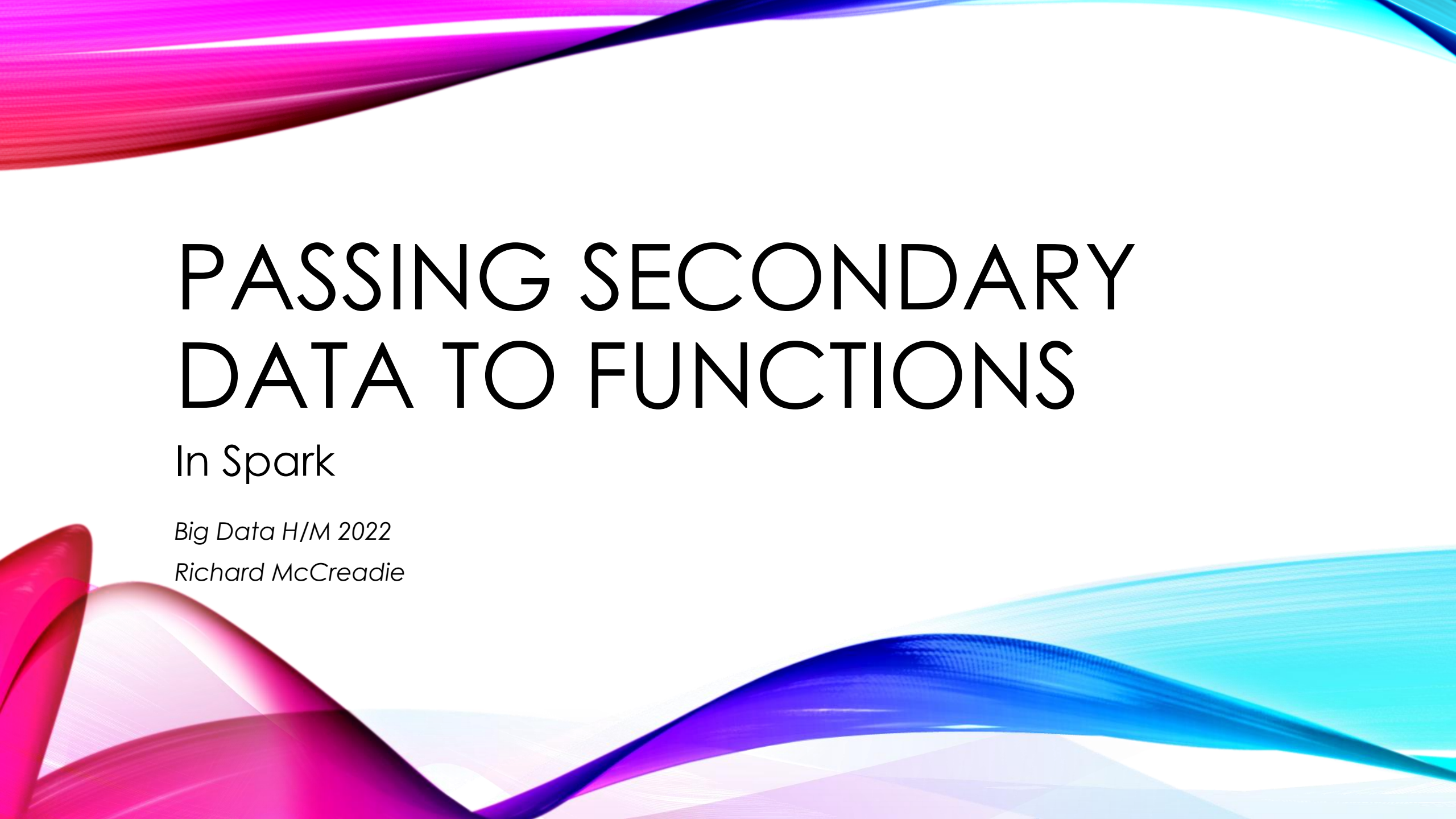




PASSING SECONDARY DATA TO FUNCTIONS

In Spark

Big Data H/M 2022

Richard McCreadie



MOTIVATION

- We have covered the main transformation functions that you will need to tackle your assessed exercise, however, there are a couple of extra topics that are useful to cover
- The first of these is [how to pass data to these transformation functions outside of the input they are transforming](#)
- Example
 - Assume that we are processing a set of student exam results and we want to calculate the band for each exam result using a MapFunction. We have a separate data structure that tells us how mark ranges convert to bands, how do we pass this data structure to the MapFunction?

JOINS

(I.E. THE STUPID WAY)

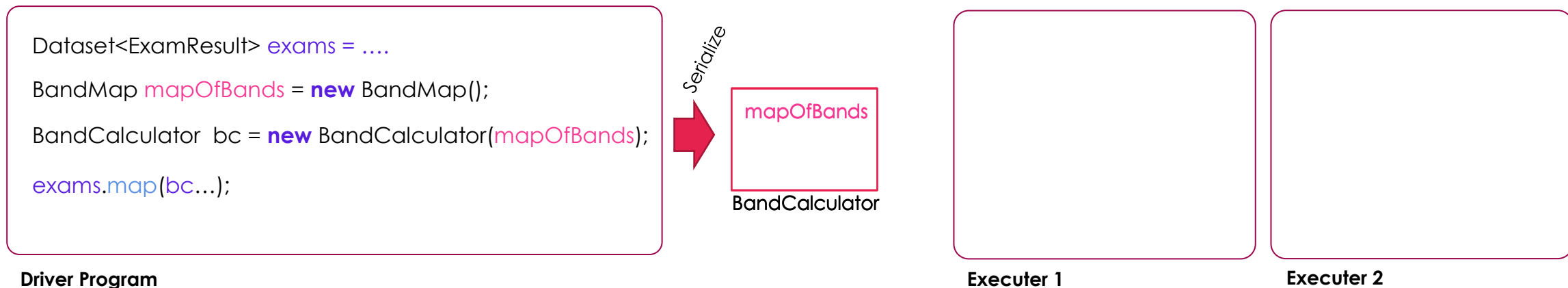
- The worst way we could do this is to attach a copy of the data structure to each student exam result, i.e.

```
Dataset<ExamResult> -> Dataset<Tuple2<ExamResult, BandMap>>
```

- This is of course bad, since we are creating a new copy of BandMap for each ExamResult, which is very wasteful of resources

SERIALIZING WITH THE TRANSFORMATION FUNCTION

- The second option is to make use of how Spark sends transformation functions to the executors that will use them
- Spark constructs transformation functions at the driver program, serializes them, and copies them to each executor



BROADCAST VARIABLES

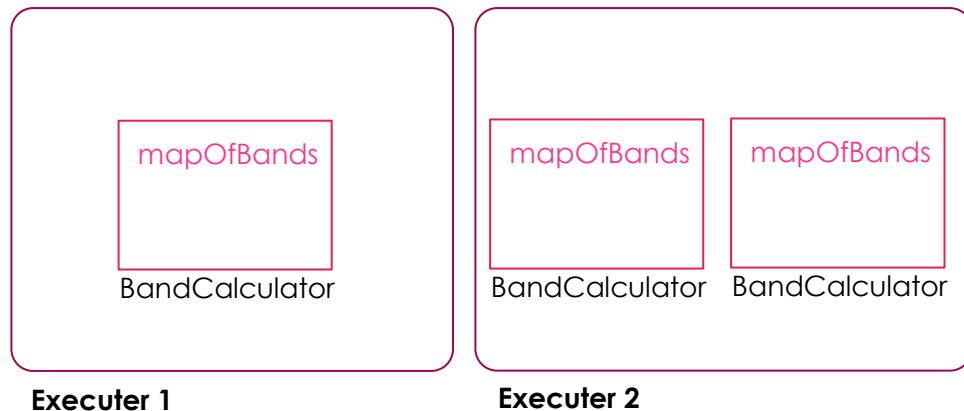
- Serialization of data with a transformation function works well so long as we only need the additional data structure for a single function, but what if we have multiple functions that need the same data structure?
- This is the role of a **broadcast variables**
 - Broadcasting takes as input a Java object to be shared
 - This creates a Broadcast Variable, which is like a pointer to the real data
 - At execution time, sends a copy of all broadcasted data to all executors, where the broadcasted copy can be referred to via the Broadcast variable

```
JavaSparkContext javaSparkContext = JavaSparkContext.fromSparkContext(spark.sparkContext());  
BandMap mapOfBands = new BandMap();  
Broadcast<BandMap> mapOfBandsBV = javaSparkContext.broadcast(mapOfBands);  
BandCalculator bc = new BandCalculator(mapOfBandsBV);
```

BROADCAST VARIABLES

- The key difference between broadcast variables and variables serialized by transformation functions is that a broadcast variable is just a pointer to the real data, rather than a copy of that data

Serialized by Transformation Functions



Broadcast Variables

